

Web3 Address Tracker Chat App

"Telegram-bot in a browser"

Standalone product, separate from Quify | v1 Scope | April 2026

A web app where users chat with a bot that tracks Ethereum/L2 wallet addresses in real time, using inline buttons (Telegram-style) instead of free-form typing. Users sign in via email/social, watch wallets, and follow curated whale wallets -- all inside a persistent chat thread.

1. Context

Why:

Telegram address-tracker bots (Whale Alert, DeBank-style trackers, smart-money copy bots) are extremely popular but locked inside Telegram. That excludes users who do not use Telegram, makes onboarding clunky (BotFather, /start, group setup), and gives the operator no control over UX, branding, or monetization.

What:

A standalone web app with a chat-style UI. Users sign in (email/social), and a single in-app bot lets them watch wallets and follow whales. The bot replies with rich messages plus inline buttons so 90% of interactions are taps, not typing. Alerts are delivered as new bot messages in the chat thread.

Outcome:

A faster, prettier, web-native version of the Telegram tracker pattern -- one app, one login, no Telegram required, controlled by us end-to-end.

2. Goals & Non-Goals

Goals (v1)

- Real-time wallet watchlist with in-chat alerts on inbound/outbound activity.
- Whale/smart-money following -- curated list, subscribe with one tap.
- Telegram-bot-style chat UX: inline button rows drive all interactions.
- Multi-chain reads: Ethereum mainnet, Base, Arbitrum.
- Persistent per-user state (watchlists, alert prefs) via account login.

Non-goals (v1)

- No natural-language / LLM bot. Button-driven only; free-text for address search.
- No transaction signing, no wallet connect, no on-chain writes. Read-only.

- No mobile native app. Responsive web only.
- No email / push / Telegram-forwarding alerts. In-chat only.
- No Solana / non-EVM chains.
- No social features (sharing, comments, leaderboards).

3. Target User

A retail crypto user / on-chain analyst who currently uses Telegram bots like @WhaleAlertBot or smart-money trackers and wants:

1. A cleaner web UI with persistent history.
2. Multi-wallet watchlists across ETH + L2s in one place.
3. Curated whale lists they can opt into without finding addresses themselves.

4. Core User Stories (v1)

1. As a new user, I sign in with email or Google and immediately see the bot greet me with buttons: **[Add wallet]** **[Browse whales]** **[How it works]**.
2. As a user, I tap **[Add wallet]**, paste 0x... or vitalik.eth, and the bot replies with a wallet summary card (balance, top tokens, recent tx count) plus buttons: **[Watch]** **[View txs]** **[Holdings]** **[Cancel]**.
3. As a user, I tap **[Watch]** -- the wallet enters my watchlist and the bot confirms with threshold info and **[Change threshold]** **[Unwatch]** buttons.
4. As a user, when a watched wallet has qualifying activity, the bot posts an alert message with **[View tx]** **[View wallet]** **[Mute 1h]** buttons.
5. As a user, I tap **[Browse whales]** and see a paginated curated list -- each entry has **[Follow]** **[Preview]**. Following = same alert pipeline as a manual watch.
6. As a user, I open the app a day later -- chat history is intact and unread alerts are badged.

5. UX -- Chat & Inline Buttons

Layout (responsive web)

- Left rail: chat list (v1 has one thread: "Tracker Bot"). Future: per-topic threads.
- Main pane: scrollable message list. Each bot message can include:
 - Markdown text + emoji
 - A card block (wallet preview, tx detail, whale entry) -- title, key/value rows, optional thumbnail
 - Up to 3 button rows (max 3 buttons/row) -- each button calls a known intent
- Bottom: thin input that only accepts a wallet address or ENS (search/add shortcut).

Interaction model

- Each bot message stores its `intents[]` (button -> action). Clicking a button dispatches the intent server-side, which returns the next bot message.
- Mirrors Telegram's `InlineKeyboardMarkup` + `callback_query` pattern.
- All bot replies are deterministic (no LLM): templated views over Alchemy data.

6. Feature Detail

6.1 Watchlist

- Up to 20 wallets per user (v1 cap).
- Per-wallet config: enabled chains (ETH/Base/Arb), min USD threshold (default \$100), mute-until timestamp.
- Source of truth: Supabase row per (user, address).

6.2 Real-time Alerts

- Triggered by **Alchemy Notify** (Address Activity webhooks), one webhook per chain pointing to a Next.js route handler.
- On receipt: match watching users, apply threshold + mute, insert `alert` row, push via Supabase Realtime.
- Each alert renders as a bot message with `[View tx]` `[View wallet]` `[Mute 1h]` `[Unwatch]` buttons.

6.3 Whale / Smart-money List

- Curated table (~50 entries in v1): known funds, top DEX traders, NFT whales.
- "Follow" = add to watchlist tagged `source=whale`. Counts toward the 20-wallet cap.
- Editable via a simple admin route. No automatic discovery in v1.

6.4 Wallet Summary Card

- ETH/native balance per chain -- `alchemy_getBalance` / portfolio API.
- Top 5 ERC-20 holdings by USD -- `alchemy_getTokenBalances` + price source.
- NFT count summary -- `getNFTsForOwner` (count only, no gallery in v1).
- Last 10 txs across selected chains -- `alchemy_getAssetTransfers`.

6.5 Auth

- **Firestore Auth** -- already integrated; supports email/password, Google, and other providers out of the box.

- User document in Firestore `users/{uid}` keyed by Firebase uid.

7. Technical Architecture

Stack: Next.js 15 (App Router) on Vercel, **Firebase Auth + Firestore** (existing), Alchemy SDK + Notify webhooks. TypeScript end-to-end.

Repo layout (new standalone repo)

```
tracker-app/  
  app/  
    (auth)/login/  
  chat/ # main chat UI  
  api/  
    webhooks/alchemy/ # POST handler for Alchemy Notify  
    intents/ # POST handler for button-click intents  
  components/chat/ # Message.tsx, ButtonRow.tsx, cards/  
  lib/  
    firebase.ts # client SDK + Admin SDK init  
    alchemy.ts # SDK client + helpers  
    intents/ # one file per intent  
    alerts/dispatcher.ts # webhook -> users -> alert doc  
  firestore.rules
```

Data model (Firestore -- NoSQL)

- `users/{uid}` -- email, createdAt
- `users/{uid}/watchlist/{id}` -- address, chains[], minUsd, muteUntil, source(manual|whale), createdAt
- `whales/{id}` -- address, label, chains, description, avatarUrl, enabled
- `users/{uid}/messages/{id}` -- role(bot|user|alert), body { text, card?, buttons? }, createdAt, read
- `alerts/{id}` -- userId, watchlistId, chain, txHash, direction, valueUsd, createdAt

Data flow -- button click

1. Client POSTs to `/api/intents` with intentId + args + Firebase ID token in Authorization header.
2. Handler verifies ID token via Firebase Admin SDK, runs the intent, writes next bot message to `users/{uid}/messages`.
3. Client's existing onSnapshot listener on `users/{uid}/messages` fires automatically.

Data flow -- Alchemy webhook

1. Alchemy POSTs to `/api/webhooks/alchemy` with signed payload.
2. Verify Alchemy signing key, parse addressActivity[].

3. Collection group query across all `users/{uid}/watchlist` matching (address, chain); apply threshold + mute.
4. For each match: write `alerts/{id}` doc + `users/{uid}/messages/{id}` doc with rendered card + buttons.
5. Client's `onSnapshot` listener fires -- no extra push infra needed.

8. Critical Files to Create

Path	Purpose
<code>app/chat/page.tsx</code>	Main chat UI, subscribes to Supabase Realtime
<code>app/api/intents/route.ts</code>	Dispatches button clicks to intent handlers
<code>app/api/webhooks/alchemy/route.ts</code>	Receives and fans out Alchemy address-activity events
<code>lib/alchemy.ts</code>	Alchemy SDK wrapper (balance, tokens, txs, Notify)
<code>lib/intents/{watch,unwatch,...}.ts</code>	One handler per intent
<code>lib/alerts/dispatcher.ts</code>	Webhook payload -> user matching -> message/alert insert
<code>lib/firebase.ts</code>	Client SDK (auth + <code>onSnapshot</code>) and Admin SDK (server writes) -- two separate init exports
<code>components/chat/*.tsx</code>	Message, ButtonRow, and card rendering primitives
<code>firestore.rules</code>	Security rules: users own their subcollections; whales is public read, admin write

9. External Dependencies

- **Alchemy** -- `alchemy-sdk` for reads + Alchemy Notify for real-time webhooks. One app per chain.
- **Firebase** -- Auth + Firestore (already integrated). Real-time via `onSnapshot`. Admin SDK used server-side with a service account key set in Vercel env vars.
- **Vercel** -- Hosting + edge functions for webhook + intent handlers.
- **CoinGecko API** -- Token USD prices for threshold calculations (free tier).

10. Verification

End-to-end smoke test (manual):

1. Sign up with email -> bot greets with buttons.
2. Add vitalik.eth -> wallet card renders with non-zero ETH balance and recent txs.
3. Tap Watch -> Firestore watchlist doc created; Alchemy Notify dashboard shows address subscribed.
4. Use Alchemy webhook test tool -> alert message appears in chat within ~5s via onSnapshot.
5. Tap Mute 1h -> next test event does not produce alert. After mute expires, alerts resume.
6. Browse whales -> Follow one -> it appears in watchlist with source=whale.

Automated tests:

- Unit tests on each intent handler (mock Firebase Admin + Alchemy clients).
- Integration test on webhook dispatcher with a sample Alchemy payload fixture.
- `tsc --noEmit` and `next lint` green before shipping.

11. Open Questions (defer past v1)

- Pricing/monetization model (free with limits -> paid tier). Not blocking v1.
- Whale list curation process -- who edits? Simple admin route is fine for v1.
- Rate limits on intents to avoid Alchemy quota burn.
- Future: push/email/Telegram notification channels -- explicitly out of v1.
- Future: portfolio analytics, P&L, NFT galleries, on-chain copy-trading -- out of v1.

12. Alchemy SDK -- Setup & API Reference

Installation & Initialization

Install the SDK:

```
npm install alchemy-sdk
```

Create one Alchemy instance per chain:

```
import { Alchemy, Network } from 'alchemy-sdk';

const ethAlchemy = new Alchemy({ apiKey: process.env.ALCHEMY_API_KEY_ETH, network:
Network.ETH_MAINNET });
const baseAlchemy = new Alchemy({ apiKey: process.env.ALCHEMY_API_KEY_BASE, network:
Network.BASE_MAINNET });
const arbAlchemy = new Alchemy({ apiKey: process.env.ALCHEMY_API_KEY_ARB, network:
Network.ARB_MAINNET });
```

Required env vars (one Alchemy app per chain, created in the Alchemy dashboard):

```
ALCHEMY_API_KEY_ETH=
ALCHEMY_API_KEY_BASE=
ALCHEMY_API_KEY_ARB=
ALCHEMY_WEBHOOK_SIGNING_KEY= # from Notify dashboard
ALCHEMY_WEBHOOK_ID_ETH=wh_xxx
ALCHEMY_WEBHOOK_ID_BASE=wh_yyy
ALCHEMY_WEBHOOK_ID_ARB=wh_zzz
```

getBalance -- Native ETH balance

Signature:

```
getBalance(addressOrName: string, blockTag?: BlockTag): Promise<BigNumber> // result in wei
```

Example:

```
const wei = await alchemy.core.getBalance('vitalik.eth');
const eth = Number(wei) / 1e18;
```

- Returns BigNumber in wei. ENS names resolved automatically.
- Use blockTag = "latest" (default) for current balance.

getTokenBalances -- ERC-20 holdings

Key overloads:

```
// All ERC-20s ever held (paginated)
getTokenBalances(address: string): Promise<TokenBalancesResponseErc20>

// Top 100 tokens by 24h volume (mainnet only -- fastest for wallet card)
getTokenBalances(address, { type: TokenBalanceType.DEFAULT_TOKENS }):
Promise<TokenBalancesResponse>

// Specific contract addresses only
getTokenBalances(address, contractAddresses: string[]): Promise<TokenBalancesResponse>
```

Example:

```
import { TokenBalanceType } from 'alchemy-sdk';

// Fast: top-100 for wallet summary card
const top = await alchemy.core.getTokenBalances(address, {
  type: TokenBalanceType.DEFAULT_TOKENS,
});
// top.tokenBalances: Array<{ contractAddress, tokenBalance }>
// top.pageKey -- pass to next call for pagination
```

- DEFAULT_TOKENS (top-100 by volume) is fastest -- use for wallet summary cards.

- Use ERC20 type + pagination only when user explicitly requests full holdings.

getAssetTransfers -- Transaction / transfer history

Parameters:

```
interface AssetTransfersWithMetadataParams {  
  category: AssetTransfersCategory[]; // REQUIRED:  
  'external'|'internal'|'erc20'|'erc721'|'erc1155'  
  fromAddress?: string;  
  toAddress?: string;  
  fromBlock?: string; // hex string, default '0x0'  
  toBlock?: string; // hex string, default 'latest'  
  contractAddresses?: string[];  
  excludeZeroValue?: boolean; // default true -- filters spam/dust  
  maxCount?: number; // default 1000, max 1000  
  withMetadata?: boolean; // true = include gas, logs, etc.  
  order?: 'asc' | 'desc';  
  pageKey?: string; // for pagination  
}
```

Example (last 10 txs for wallet summary card):

```
const result = await alchemy.core.getAssetTransfers({  
  fromAddress: '0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045',  
  category: ['external', 'internal', 'erc20'],  
  order: 'desc',  
  maxCount: 10,  
  withMetadata: true,  
});  
// result.transfers: Array<AssetTransfer>  
// result.pageKey -- present if more pages exist
```

Example (detect inbound activity for alert):

```
const result = await alchemy.core.getAssetTransfers({  
  toAddress: watchedAddress,  
  category: ['external', 'erc20', 'erc721'],  
  fromBlock: lastCheckedBlock,  
  excludeZeroValue: true,  
});
```

getNftsForOwner -- NFT count summary

Signature:

```
getNftsForOwner(owner: string, options?: GetNftsForOwnerOptions):  
Promise<OwnedNftsResponse>
```



```
interface OwnedNftsResponse {
  ownedNfts: OwnedNft[];
  pageKey?: string;
  totalCount: number; // total NFTs owned -- use this for the card count
  validAt: { blockNumber, blockHash };
}
```

Example (count only for wallet summary card):

```
import { NftExcludeFilters } from 'alchemy-sdk';

const result = await alchemy.nft.getNftsForOwner('vitalik.eth', {
  excludeFilters: [NftExcludeFilters.SPAM],
});
console.log(result.totalCount); // e.g. 47 -- render as '47 NFTs'
```

- Only use totalCount in v1. Full NFT gallery is out of scope.

Alchemy Notify -- Address Activity Webhooks

Dashboard setup:

1. Go to the Notify tab in your Alchemy app dashboard.
2. Create a new Address Activity webhook -- set URL to your deployed `/api/webhooks/alchemy` endpoint.
3. Copy the Signing Key -- store as `ALCHEMY_WEBHOOK_SIGNING_KEY` env var.
4. Add addresses via dashboard or programmatically (see below).

Programmatic address registration:

```
import { Alchemy, Network } from 'alchemy-sdk';

const alchemy = new Alchemy({ apiKey: process.env.ALCHEMY_API_KEY_ETH, network:
Network.ETH_MAINNET });

// Add addresses to an existing webhook
await alchemy.notify.addAddresses(process.env.ALCHEMY_WEBHOOK_ID_ETH!, ['0x...']);

// Remove addresses
await alchemy.notify.removeAddresses(process.env.ALCHEMY_WEBHOOK_ID_ETH!, ['0x...']);

// List all webhooks
const webhooks = await alchemy.notify.getAllWebhooks();

// Get addresses on a webhook (paginated)
const addresses = await alchemy.notify.getAddresses(webhookId, { limit: 100, after:
pageKey });
```

Webhook payload shape (Address Activity):

```
{
  webhookId: string,
  id: string,
  type: 'ADDRESS_ACTIVITY',
  event: {
    network: 'ETH_MAINNET' | 'BASE_MAINNET' | 'ARB_MAINNET',
    activity: Array<{
      fromAddress: string,
      toAddress: string,
      blockNum: string, // hex
      hash: string, // tx hash
      value?: number, // ETH value (native transfers)
      asset?: string, // token symbol e.g. "USDC"
      category: 'external' | 'internal' | 'erc20' | 'erc721' | 'erc1155',
      rawContract?: {
        value: string, // hex token amount
        address: string, // token contract address
        decimal: string, // hex decimals
      },
    }>
  }
}
```

Signing key verification (Next.js route handler):

```
// app/api/webhooks/alchemy/route.ts
import { createHmac } from 'crypto';
import { NextRequest, NextResponse } from 'next/server';

export async function POST(req: NextRequest) {
  const rawBody = await req.text();
  const signature = req.headers.get('x-alchemy-signature') ?? '';
  const expected = createHmac("sha256", process.env.ALCHEMY_WEBHOOK_SIGNING_KEY!)
    .update(rawBody).digest("hex");

  if (expected !== signature)
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const payload = JSON.parse(rawBody);
  await dispatchAlchemyActivity(payload); // -> Firestore writes
  return NextResponse.json({ ok: true });
}
```

- Header name: x-alchemy-signature (HMAC-SHA256 of the raw request body, NOT parsed JSON).

Utility -- getWalletSummary (reusable helper)

Combine all three read APIs into one function, reused by intents and webhook enrichment:

```
// lib/alchemy.ts
export async function getWalletSummary(address: string, chains: Chain[]) {
  return Promise.allSettled(
    chains.map(async (chain) => {
      const sdk = sdkForChain(chain);
      const [balanceWei, tokens, nfts, transfers] = await Promise.all([
        sdk.core.getBalance(address),
        sdk.core.getTokenBalances(address, { type: TokenBalanceType.DEFAULT_TOKENS }),
        sdk.nft.getNftsForOwner(address, { excludeFilters: [NftExcludeFilters.SPAM] }),
        sdk.core.getAssetTransfers({
          fromAddress: address, category: ['external', 'internal', 'erc20'],
          order: 'desc', maxCount: 5,
        }),
      ]);
      return { chain, balanceWei, tokens, nfts, transfers };
    })
  );
}
```